

How to Work with AI Without Losing Control

The Problem Most People Hit

AI tools are genuinely useful. In the early stages of any project, they feel almost magical. You describe what you need, and something useful appears within seconds.

Then the project grows.

You have dozens of conversations across multiple tools. You have documents drafted in one chat, revised in another, and approved in a third. You cannot remember which version is current. You are not sure what decisions were made or why. A team member asks a question and you realise the answer is buried somewhere in a chat history you cannot search.

This is not a failure of the AI tools. It is a structural problem. Ad hoc AI use works well for small, isolated tasks. It breaks down when the work accumulates.

The fix is not complicated. It is structure applied early.

[!NOTE]

The single most common failure in AI-assisted projects is not a bad AI output. It is the absence of a system for managing what AI produces.

Slow Down Early to Move Faster Later

Every hour spent organising your approach before a project grows complex saves several hours of rework afterwards.

This is not a new idea. It applies to any complex work. What is different with AI tools is how quickly complexity accumulates. An AI can produce in minutes what a human might take days to write. That speed is an advantage. It is also a risk, because the volume of output grows faster than most people's ability to manage it.

The practices in this framework are lightweight. They add a small amount of upfront discipline. In return, they prevent the situations that cost real time: hunting for the right version of a document, re-explaining decisions that were never recorded, and discovering that two parts of a project contradict each other.

Why Version Control Matters

When you save a document to a chat window or a local folder without any versioning discipline, you have one version. When you update it, the old version disappears.

Version control gives you a complete history of every change, who made it, and when. It means you can always go back. It means you can see exactly what changed between one version and the next. It means when someone asks “why did we decide that?” you have an answer.

GitHub is one tool for version control. There are others. The tool matters less than the discipline of using one consistently.

[!NOTE]

Version control is not just for software developers. Anyone managing documents, plans, or requirements in a complex project benefits from it.

Why Document Ownership Matters

Every document in an AI-assisted project should have a named human owner. Not an AI tool. A person.

The owner is responsible for the accuracy of the document. The owner decides when it is ready to share. The owner approves final versions.

AI tools draft, revise, review, and suggest. Humans decide and own.

This distinction matters for two reasons. First, AI tools make mistakes. They can produce content that sounds authoritative but is factually wrong. Without a human owner who has actually read and approved a document, errors go undetected.

Second, accountability is a human concept. When a client, colleague, or auditor asks who is responsible for a decision, the answer must be a person.

Why Large Projects Need Modular Documentation

A single large document covering everything about a project is appealing at the start. It feels organised. In practice, it becomes impossible to manage as the project grows.

Modular documentation means keeping a master document that covers scope, objectives, and principles at a high level, and separate documents for each capability or component. Each module document covers one thing in detail.

This approach has several benefits:

- Different people or AI tools can work on different modules without stepping on each other.
- Reviews are focused. A reviewer of the customer module only needs to understand that module.
- When something changes, you update one module, not one enormous document.
- AI tools perform better when given focused, bounded context rather than sprawling documents.

[NOTE]

The master document should never become the dumping ground for every detail. Its job is to hold the overall picture and link to the detail that lives in module documents.

Why Multiple AI Tools Can Be Useful

Different AI tools have different strengths. One tool might be stronger at long-form document drafting. Another might be stronger at technical analysis. A third might surface considerations the others missed.

More practically: using two AI tools on the same problem and comparing their outputs is a useful quality check. When they agree, you have higher confidence. When they disagree, that disagreement is information. It often reveals an ambiguity in your brief, an assumption that has not been made explicit, or a genuine complexity that needs your attention.

This is similar to a four-eyes principle in audit and financial controls. No single party both produces and reviews their own work.

Assign different roles to different tools. Keep those roles explicit. The framework document on collaboration covers this in more detail.

Why AI Disagreement Can Be Valuable

When two AI tools give different answers to the same question, most people treat it as a problem. It is worth treating it as an opportunity.

Disagreement surfaces ambiguity. If the question was clear and unambiguous, two capable tools should produce similar answers. When they diverge, the question was probably underspecified, or the context contained competing interpretations.

The right response to AI disagreement is not to pick the answer you prefer. It is to understand why they disagreed, clarify the question, and then decide.

Why Issues Should Be Tracked, Not Ignored

Every project accumulates unresolved questions. Some are blockers. Most are not.

The mistake is letting non-blocking issues disappear into chat history on the assumption that someone will deal with them eventually. They reappear later as surprises, usually at the worst possible time.

An issues register is a simple list. Each entry has a title, a status, and an owner. Items move from open to resolved as they are dealt with. Items that are accepted as known risks get recorded as such.

Tracking issues does not mean resolving everything before you can proceed. It means knowing what you have chosen not to resolve yet, and why.

[!NOTE]

Not every issue is a blocker. Some issues can be accepted, tracked, and resolved later.

What matters is that they are visible, not hidden.

Why Human Accountability Must Stay Explicit

As AI tools become more capable, there is a gradual tendency to treat their outputs as reliable without review. This is a mistake.

AI tools can produce incorrect information with complete confidence. They can misinterpret a brief, apply the wrong context, or fabricate a reference that does not exist. None of this is malicious. It is a property of how these tools work.

The safeguard is human review. Every output that will be used to make a decision, shared with a client, or built upon by further work must be reviewed by a human who takes responsibility for it.

This is not a commentary on whether AI tools are good or bad. It is a practical observation: accountability requires a human in the loop, and that human must actively review, not passively accept.

Summary: The Practices That Make a Difference

These are the habits that prevent AI-assisted work from becoming unmanageable:

Practice	Why It Matters
Use version control	Preserve history, enable rollback, track change
Name a human owner for every document	Accountability requires a person
Use modular documentation	Keep detail manageable and focused
Assign explicit roles to AI tools	Clarity prevents overlap and gaps
Track unresolved issues	Visibility prevents late surprises
Record material decisions	Decisions need to be understood later
Review every AI output before relying on it	AI tools make mistakes
Do not let AI resolve uncertainty silently	Ambiguity needs human direction

None of these practices are burdensome on their own. Together, they create a foundation that lets AI tools be genuinely useful over the life of a project, not just at the start.

Version History

Version	Date	Author	Summary
0.1	2026-05-28		Initial draft
1.0	2026-06-13		Initial Version (Approved)